Design note: Never Lose a Heartbeat

Architecture

- **Collector** (device thread). Reads sensor 1 and 2 from the mock every 0.3 s (immediately again while a reply is full). Every reply becomes one SQLite transaction (WAL, `synchronous=FULL`): samples, loss records, interval bookkeeping. One row per sample, key `(session, sensor, index)`, so “exactly once” is enforced by the database.
- **Heartbeat** (device thread). `GET /health` every second (2 s timeout). 2 failures => OFFLINE, 2 successes => ONLINE. Measured: offline detected in 0.1 s (refused) to 4.1 s (silent drop); back online in 1.2 to 3.2 s. Limit is 10 s.
- **Controller** (device thread). OFFLINE: computes every ready window locally (mode `offline`). ONLINE: streams new samples (+ the 10 s of context the first window needs) to the cloud, which computes the windows (mode `online`); in the background it catches up whatever the cloud lacks (oldest first, 10 000-sample gzip batches) and uploads offline results. A window is processed exactly where it first became ready; ownership is handed over with an `(epoch, own_from)` pair.
- **Cloud service** (stdlib HTTP + SQLite). Idempotent ingest (`INSERT OR IGNORE`, conflicting duplicates logged), per-session interval sets, online window processing right after each live batch.
- **Reconcile**: on every reconnect the device asks the cloud for its interval sets and replaces its own picture of the cloud, so lost ACKs, a restarted or wiped cloud, and kill -9 all converge to “send what is missing”.
- **Same logic everywhere**: `pipeline/windowing.py` is pure; device and cloud call the same function on stored samples.

Guaranteed

- Every sample the device received (CRC verified) is either committed to disk once, unchanged, or listed as a loss with sensor, index range and reason. Nothing is invented or interpolated. Gaps the mock did not report are recorded as `gap_not_reported_by_source`; disk-full becomes `device_disk_full` (a 16 MB reserve file is released to make room for the record).
- Corrupt replies (CRC mismatch) are discarded and re-read by index; duplicate replies are dropped by index; restarts close the old session with `{"to_index": null}` losses; stalls are survived by reconnecting and re-reading from our frontier.
- Mode switches do not lose or duplicate samples: the switch only changes who processes; storage is mode-independent.
- After an outage the cloud copy converges to the device copy, each sample once (2 h backlog: 61 s on loopback).
- Results follow section 3; verified against an independent naive implementation, including random gaps.
- Timing uses monotonic clocks; changing the system clock cannot reorder, drop or duplicate data.

Not guaranteed (stated honestly)

- **Power cut**: a transaction is durable once committed. Data read but not yet committed is lost: at most **1 second per sensor (one reply, <= 400 samples)**, normally < 0.3 s. After a reboot the mock is a new session, so the rest of the old session is recorded as one open-ended loss. Durability assumes the SD card/SSD honours `fsync`; cheap cards with volatile write caches can break that. I could not pull real power in my test environment (only `kill -9`).
- Online latency depends on the network: measured 0.36 s worst case on loopback, not on a real WAN/free-tier cold start.
- Free-tier clouds may sleep; the device then runs offline and catches up after wake-up.
- TLS needs a sane clock: if the clock jumps far, certificate validation can fail and the device stays offline (data stays safe).
- Disk full: data that cannot be written is lost (and listed); old data is never deleted automatically.

Assumptions

- Heart-rate “any sample missing in those 5 s” is checked on sensor 1 (set `HR_REQUIRES_SENSOR2` to change).
- Peak rule evaluated as a global greedy scan; computed on a local slice anchored where the chain cannot differ (see `windowing.py`).
- The brief does not fix the JSON shape of result lines; mean/min/max are objects keyed by sensor.
- The first window of an ONLINE period needs 10 s of sensor-1 history; the device sends it with the first live batch.

Capacity

Python process, SQLite, one core: about 3 to 8 % CPU at real-time rate on a laptop-class core; see EVIDENCE.md for measured figures.